

FULL LEGAL NAME	LOCATION (COUNTRY)	EMAIL ADDRESS	MARK X FOR ANY NON-CONTRIBUTING MEMBER
Miklos Lammel	Budapest, Hungary	lammelm@gmail.com	
Vikas Gautam	Bangalore, India	vikasgtm@gmail.com	
Gaurav Sharma	Gurgaon, India	ggsharma9555@gmail.com	

Statement of integrity: By typing the names of all group members in the text boxes below, you confirm that the assignment submitted is original work produced by the group (excluding any non-contributing members identified with an “X” above).

Team member 1	Miklos Lammel
Team member 2	Vikas Gautam
Team member 3	Gaurav Sharma

Use the box below to explain any attempts to reach out to a non-contributing member. Type (N/A) if all members contributed.

Note: You may be required to provide proof of your outreach to non-contributing members upon request.

N/A

Introduction

In this project, we explored some of the Deep Learning learning methods.

The asset we have chosen is SPY.

We downloaded data from 01-02-2010 to 31-12-2023

Models and methods implemented:

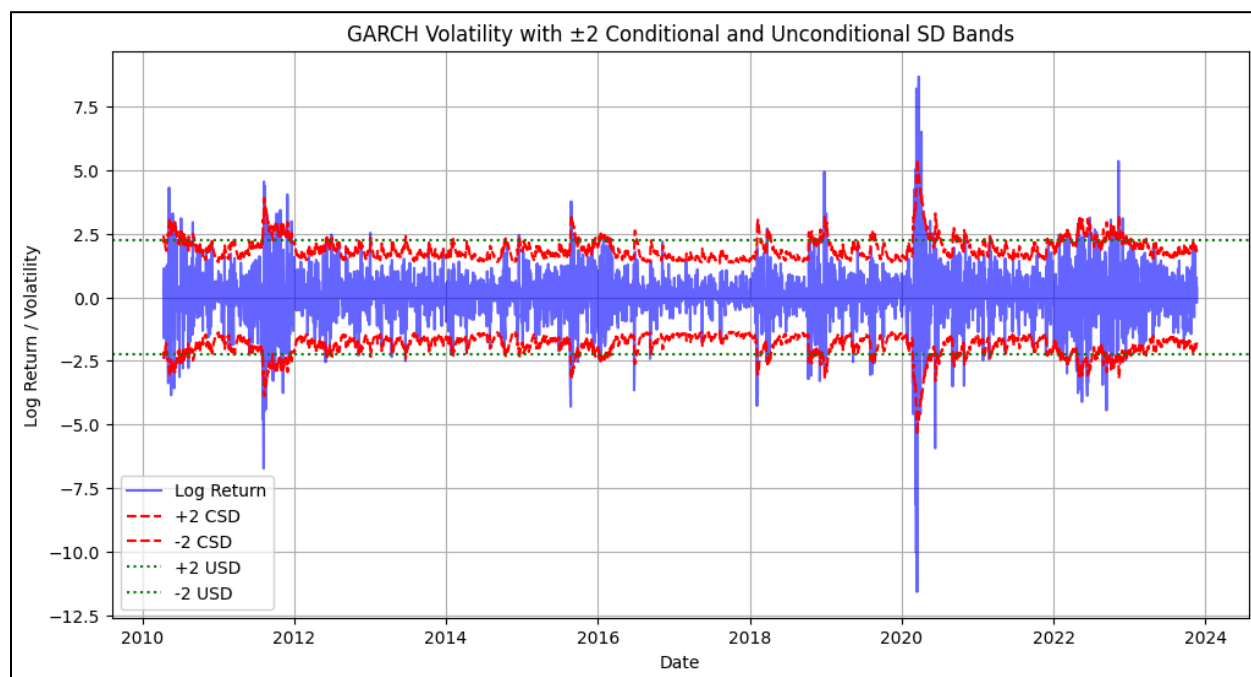
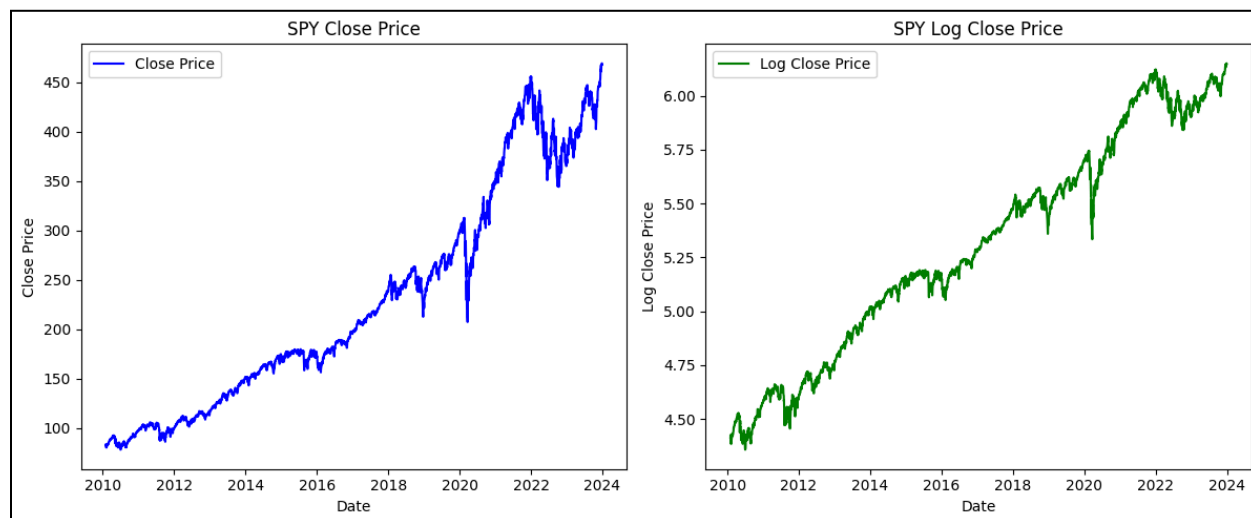
- MLP
- LSTM
- CNN-GAF
- Walk Forward

We have to see the effect with leakage of information and without leakage of information on model performance

The data downloaded has info on Close, High, low and Volume with date. For Walk Forward-LSTM we used hourly data for the same Asset.

STEP 1

Time Series Plots - Visual analysis



1. Feature engineering

We calculated Log Returns and then some other features from past lags and future lags (to show leakage of information)

Introduce features (using lags)

```
['Returnn.Lag1'] = data['Log_Return'].shift(1)
['Returnn.Lag3'] = data['Log_Return'].shift(3)
['Returnn.Lag5'] = data['Log_Return'].shift(5)
['MA_5'] = data['Log_Return'].shift(5).rolling(5).mean()
['MA_10'] = data['Close'].shift(10).rolling(10).mean()
['MA_20'] = data['Log_Return'].shift(20).rolling(20).mean()
['Rolling_Volatility5'] = data['Log_Return'].shift(5).rolling(5).std()
['Rolling_Volatility10'] = data['Log_Return'].shift(10).rolling(10).std()
['GARCH_Vol_5'] = 5*compute_garch_volatility(data['Log_Return'].shift(5).rolling(5).mean()
['GARCH_Vol_3'] = -3*compute_garch_volatility(data['Log_Return'].shift(3).rolling(3).mean()
['GARCH_Vol_51'] = 5*compute_garch_volatility(data['Log_Return'].shift(5).rolling(5).std()
['GARCH_Vol_31'] = -3*compute_garch_volatility(data['Log_Return'].shift(3).rolling(3).std()
```

Introduce features from future information (with leakage!)

```
['Future_MA_10'] = data['Log_Return'].shift(-10).rolling(10).mean()
['Future_MA_20'] = data['Log_Return'].shift(-20).rolling(20).mean()
['Future_Volatility'] = data['Log_Return'].shift(-10).rolling(10).std()
['Returnn.F.Lag1'] = data['Log_Return'].shift(-1)
['Returnn.F.Lag3'] = data['Log_Return'].shift(-3)
['Returnn.F.Lag5'] = data['Log_Return'].shift(-5)
["1-CSD"] = 1 * data["GARCH_Vol"]
["2-CSD"] = 2 * data["GARCH_Vol"]
["3-CSD"] = 3 * data["GARCH_Vol"]
["4-CSD"] = 4 * data["GARCH_Vol"]
```

Features: Define features (X) and target (y)

```
X = [["MA_5", "MA_10", "MA_20", "Rolling_Volatility5", "Rolling_Volatility10",
      'Returnn.Lag1', 'Returnn.Lag3', 'Returnn.Lag5',
      'GARCH_Vol_3', 'GARCH_Vol_5', 'GARCH_Vol_31', 'GARCH_Vol_51',
      "1-CSD", "2-CSD", "3-CSD", "4-CSD", "Spike",
      # Future information
      'Returnn.F.Lag1', 'Returnn.F.Lag3', 'Returnn.F.Lag5',
      "Future_MA_10", "Future_MA_20", "Future_Volatility"]]
```

Predicting next day's return

```
y = data['Log_Return']
```

2. Then we scaled the data and created subsequence of each feature with a window size of 15.
3. Then we train-test split the data as test data = 20% of dataset and validation data = 20% of the remaining data,,so finally we have remaining train data as 60% of total data.
4. We ran the model on train and validation data the check the performance unseen test data.

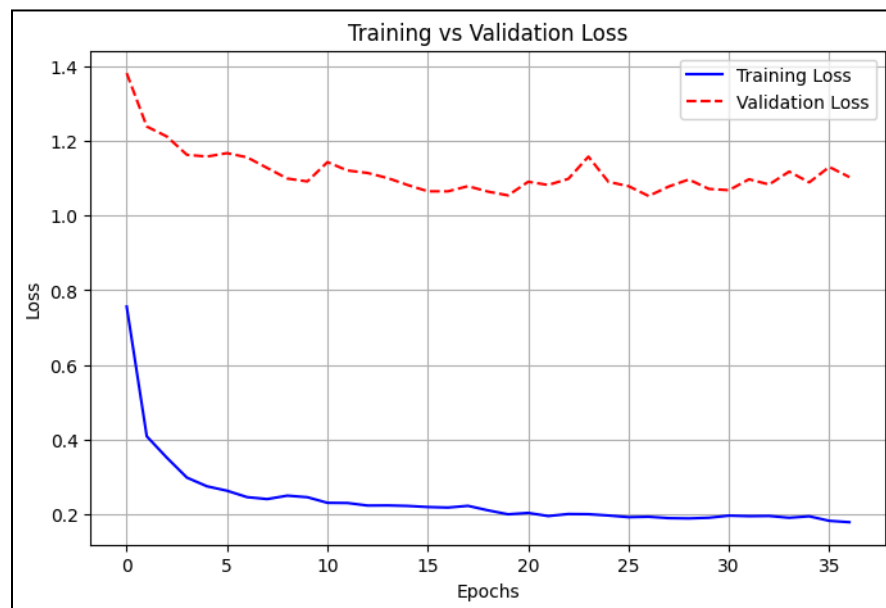
1.1 MLP

Model Architecture

Create MLP model with Dense, Dropout and Flatten and tanh as activation function

```
model1 = Sequential([  
    Flatten(input_shape=(X_train12.shape[1:])),  
    Dense(128, activation='tanh'),  
    Dropout(0.2),  
    Dense(64, activation='tanh'),  
    Dropout(0.2),  
    Dense(32, activation='tanh'),  
    Dropout(0.2),  
    Dense(16, activation='tanh'),  
    Dropout(0.2),  
    Dense(8, activation='tanh'),  
    Dropout(0.2),  
    Dense(4, activation='tanh'),  
    Dense(1) # Output layer for regression  
)
```

Model Performance



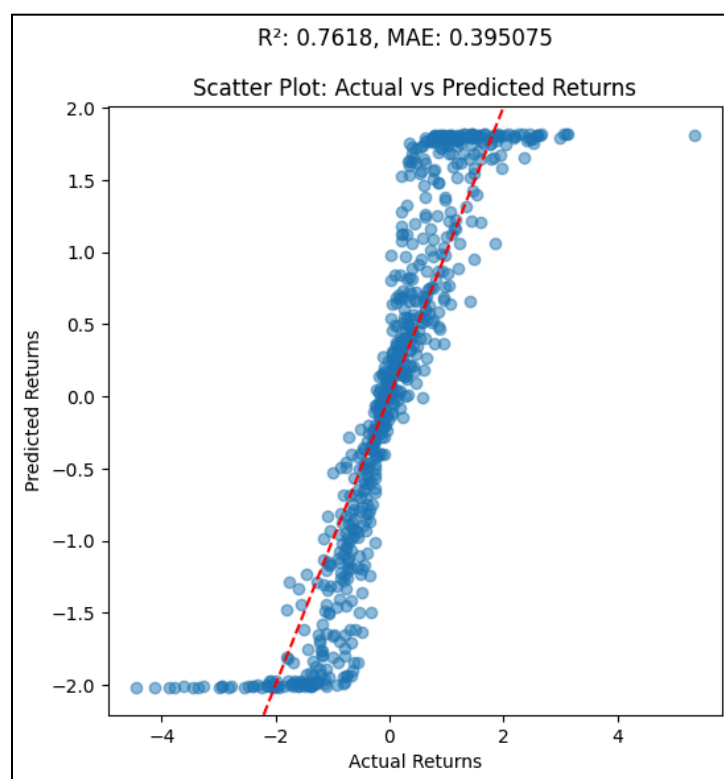
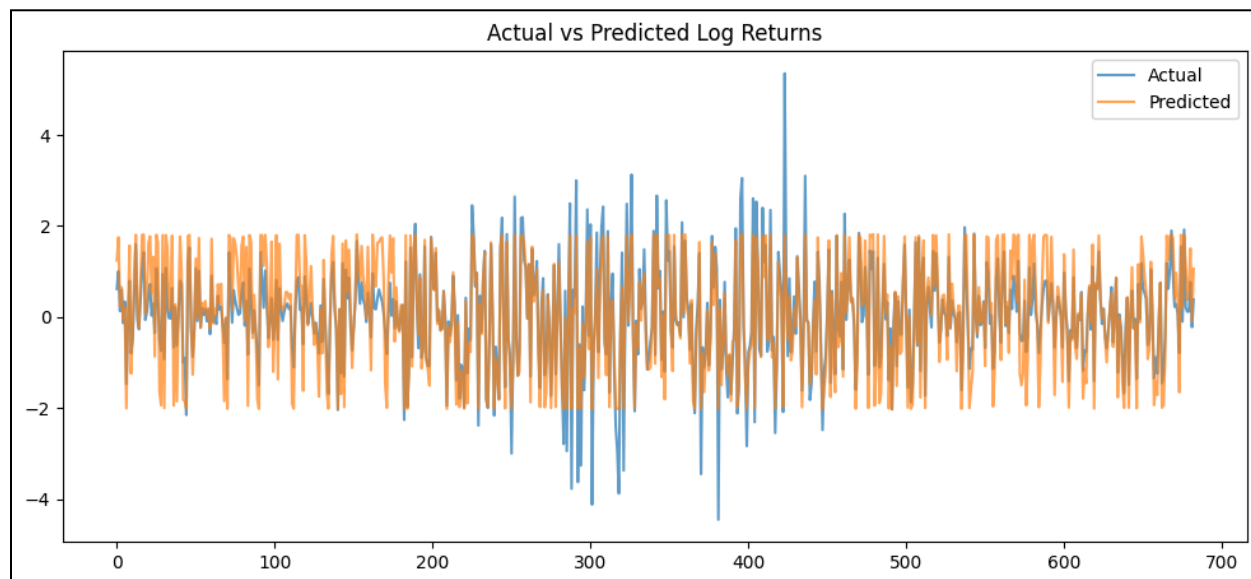
Best epoch for minimum Validation Loss: 27, Loss: 1.05

Best epoch for minimum Validation MAE: 17, MAE: 0.471

Test Data

Mean Absolute Error (MAE): 0.395

R-squared (R^2): 76.1%

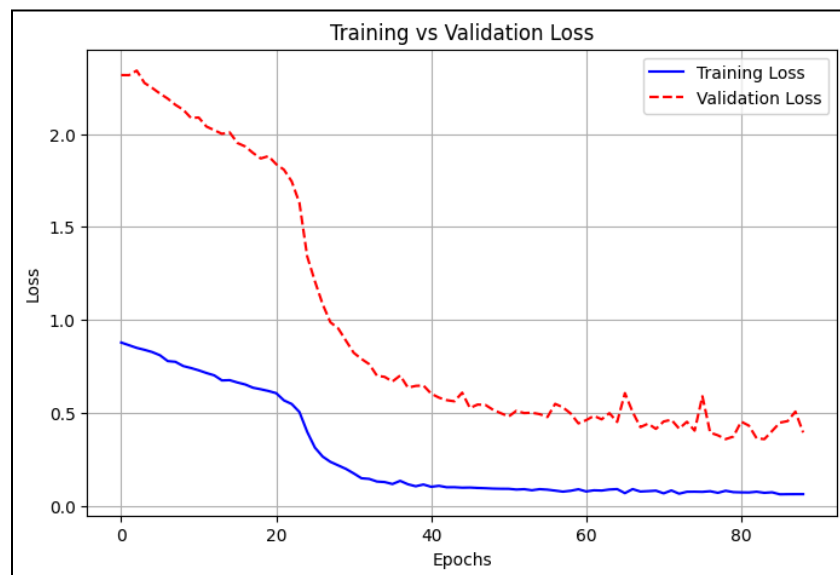


1.2 LSTM

Create LSTM model (128,64,32,16,8) with dropout and tanh as activation function

```
model2 = Sequential([
    LSTM(128, activation='tanh', return_sequences=True, input_shape=(X_train12.shape[1],
X_train12.shape[2])),
    Dropout(0.2),
    LSTM(64, activation='tanh', return_sequences=True),
    Dropout(0.2),
    LSTM(32, activation='tanh', return_sequences=True),
    Dropout(0.2),
    LSTM(16, activation='tanh', return_sequences=True),
    Dropout(0.2),
    LSTM(8, activation='tanh'),
    Dropout(0.2),
    Dense(1)
])
```

Model Performance



Best epoch for minimum Validation Loss: 79, Loss: 0.358

Best epoch for minimum Validation MAE: 77, MAE: 0.171

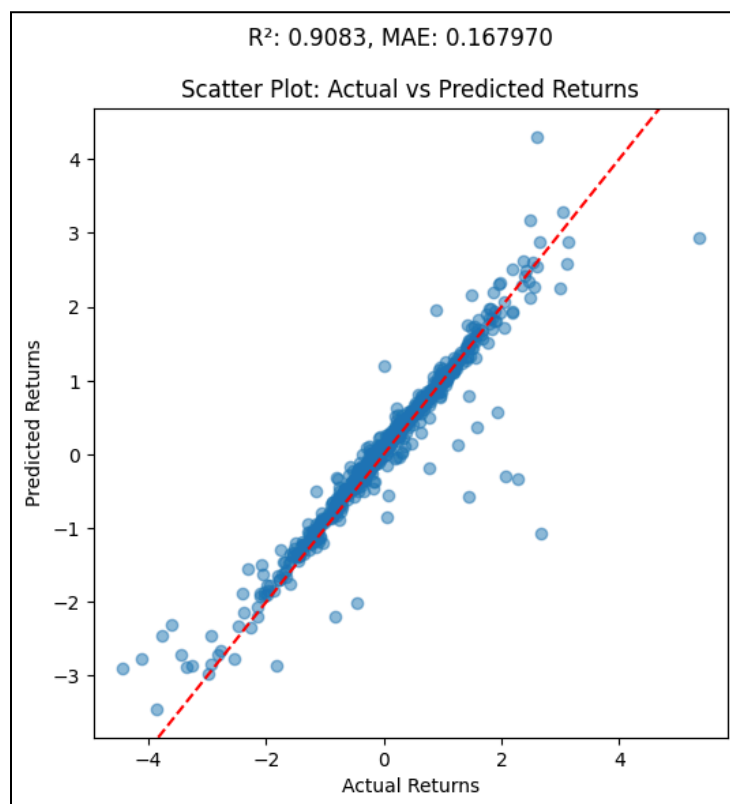
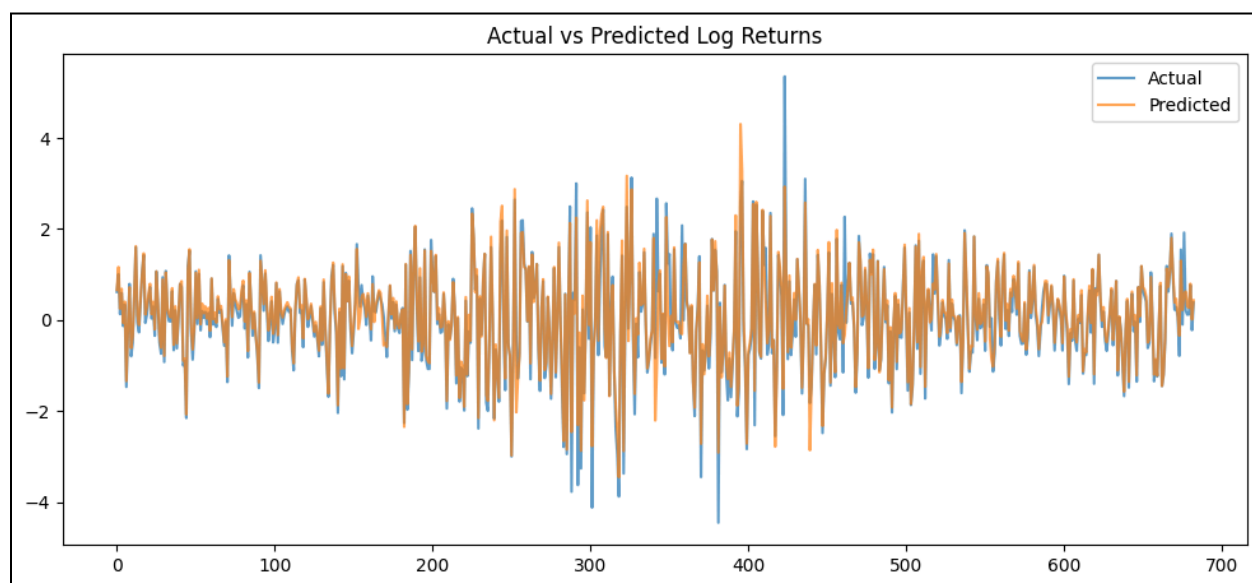
Test Data

GROUP WORK PROJECT # 3
Group Number: 8429

MScFE 642: Deep Learning for Finance

R² Score: 90%

MAE Score: 0.16



1.3 CNN-GAF

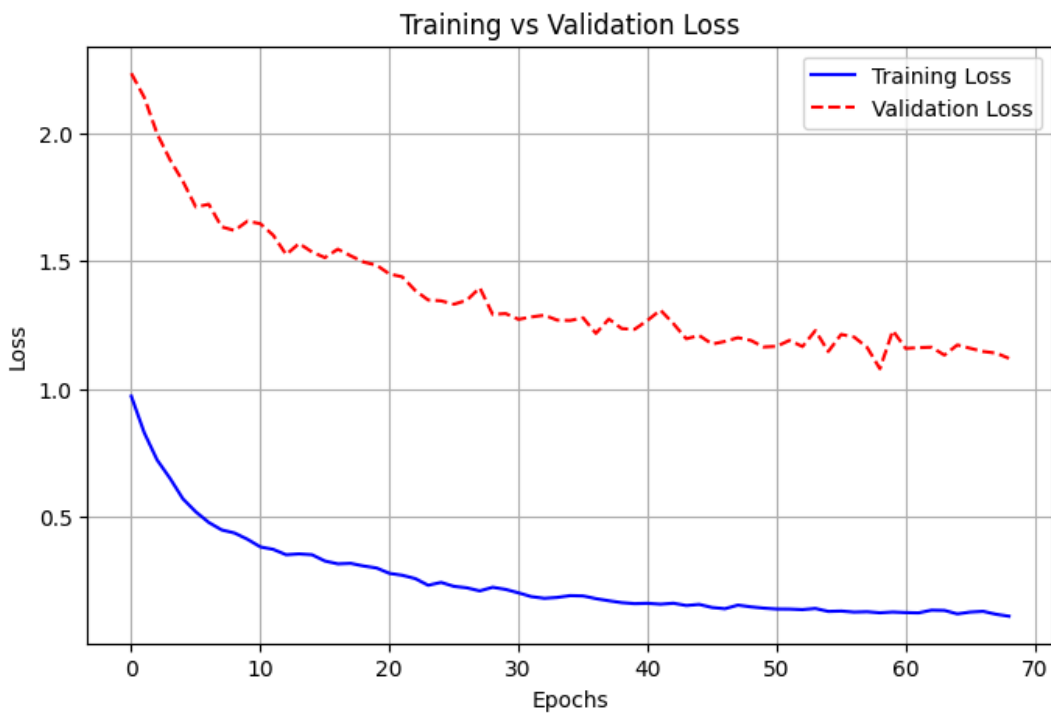
For each feature we have in the dataset, we created GAF images and then created subsequence as window size of 15. Then feed this subsequence arrays of GAF images of each feature as input features to this model. We did this step for both train and test dataset.

```
# Convert time series to GAF images
gaf = GramianAngularField(image_size=15, method='difference')
X_train_gaf = np.array([gaf.fit_transform(x.reshape(1, -1)) for x in X_train_scaled])
```

Input shape to the model from train data: (no_of_rows, 15, 15, 1)

Best epoch for minimum Validation Loss: 59, Loss: 1.078

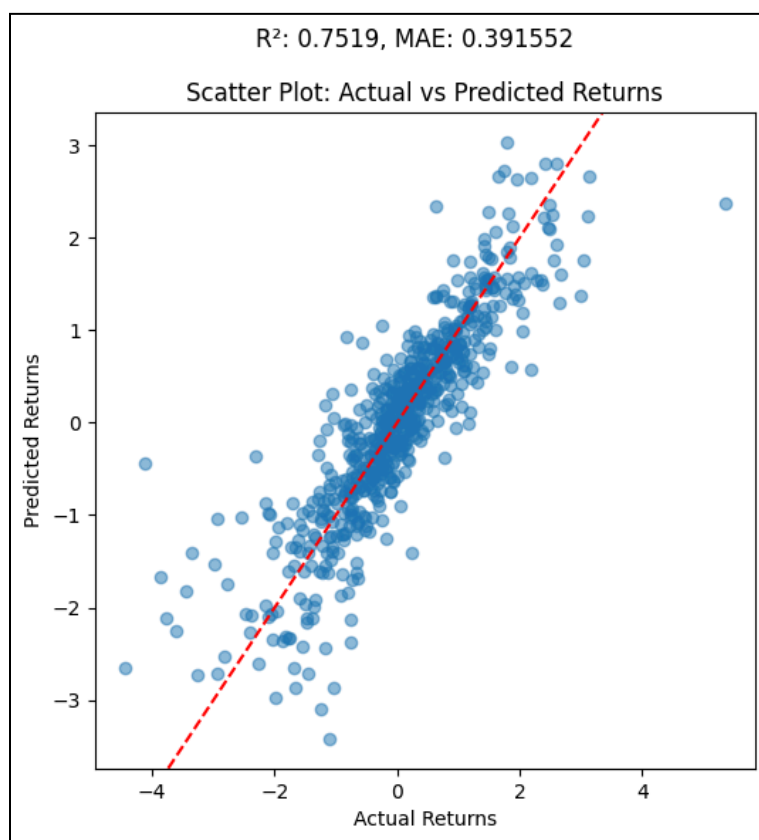
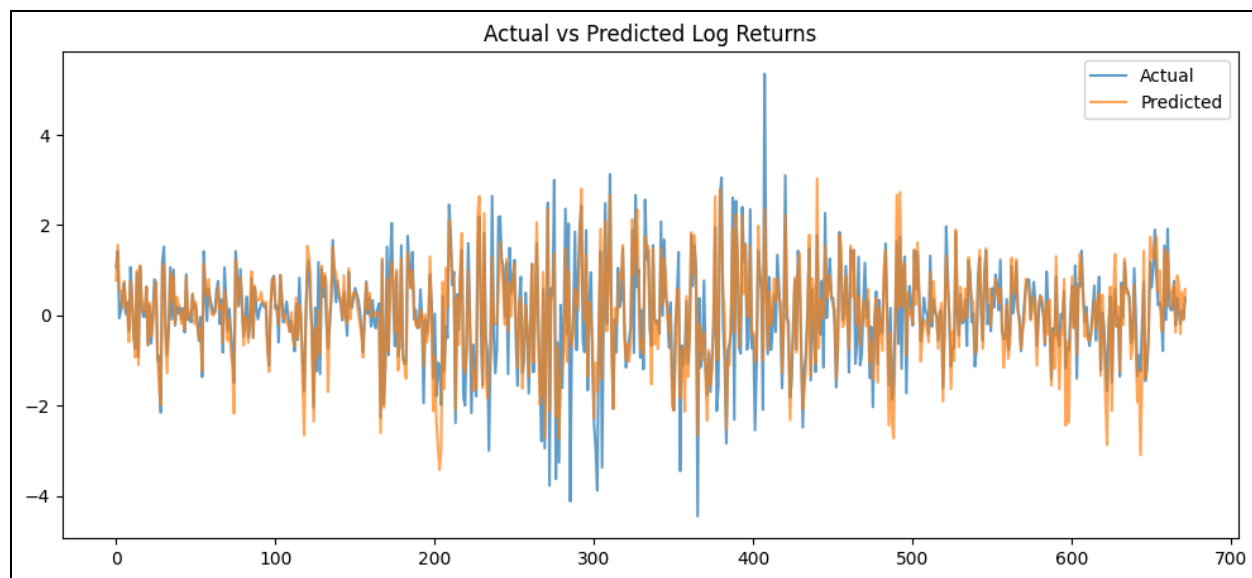
Best epoch for minimum Validation MAE: 59, MAE: 0.520



Test Data

R² Score: 75.19%

MAE Score: 0.3916



Findings

- StandardScaler is giving better R-squared than MinMaxScaler
- Method1: To create gaf images of subsequence of scaled arrays is giving lower R-squared than Method2: creating subsequence of gaf images of scaled data

Model Comparison on Test Dataset

Model	R-Squared	MAE
MLP	76.1%	0.395
LSTM	90%	0.16
CNN-GAF	75.19%	0.39

Step 2

2.1 Walk Forward MLP

We used the same MLP model as in Step 1.1: MLP

2.1.1 Train Test split: 500:500 for each set

We first scaled the data and then split into train1 and test datasets.

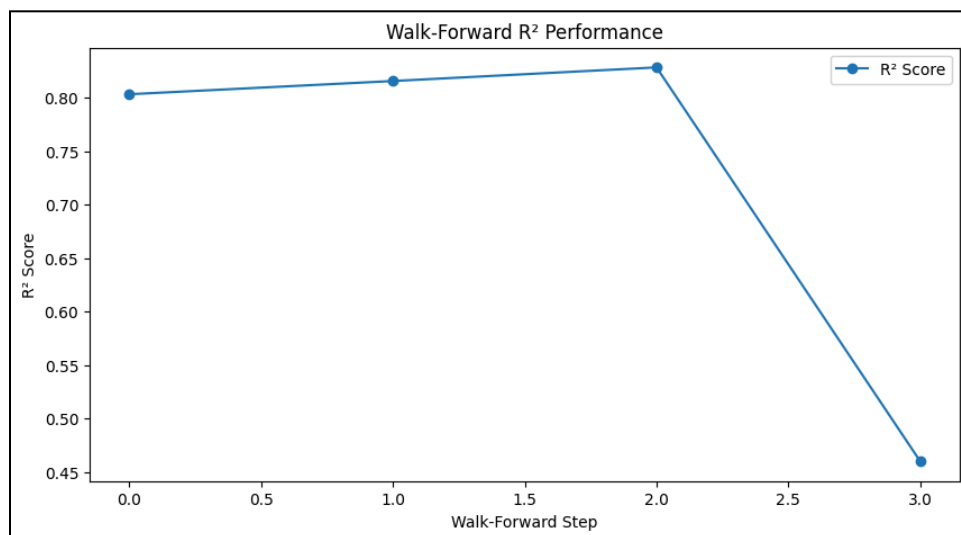
Then, We selected the first 500 records from train1 data for train data and next 500 records for test or validation data as each set for input to the model and after each iteration we move forward by 500 records as step_size.

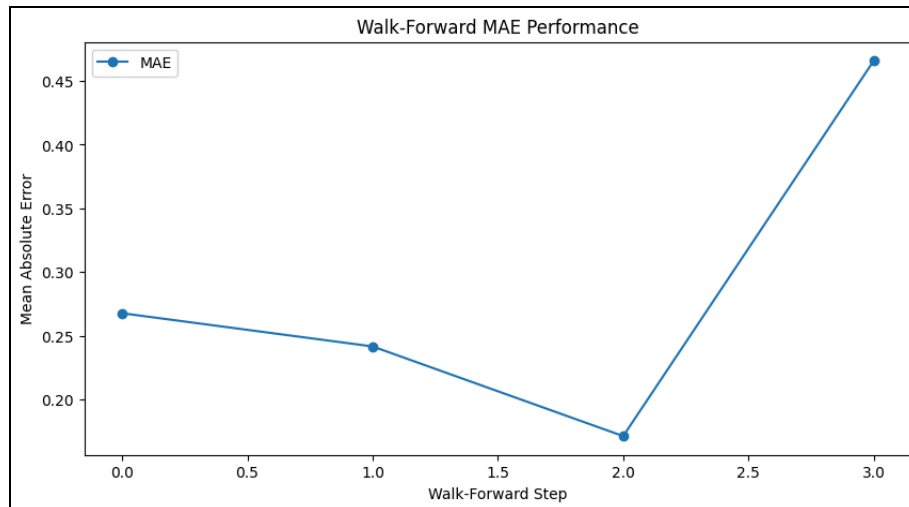
We selected the model predictions with lowest R-squared on validation data and compared the predictions on test data with actual returns.

Validation Data

Best R-squared: 0.830

Corresponding MAE: 0.236

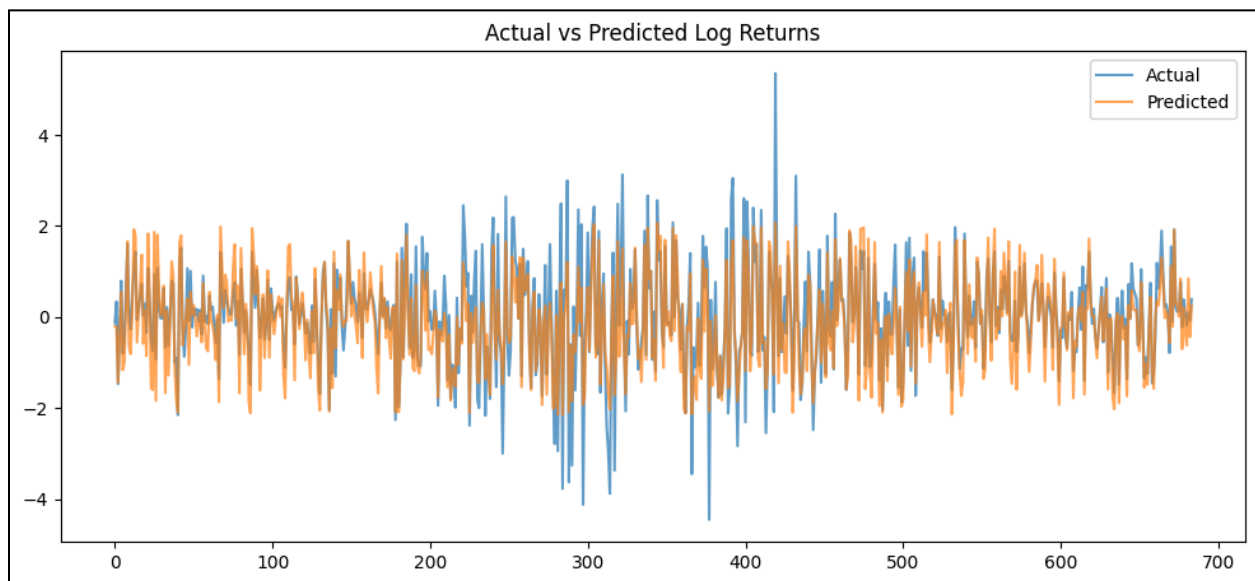




Test Data:

Mean Absolute Error (MAE): 0.390

R-squared (R^2): 0.775



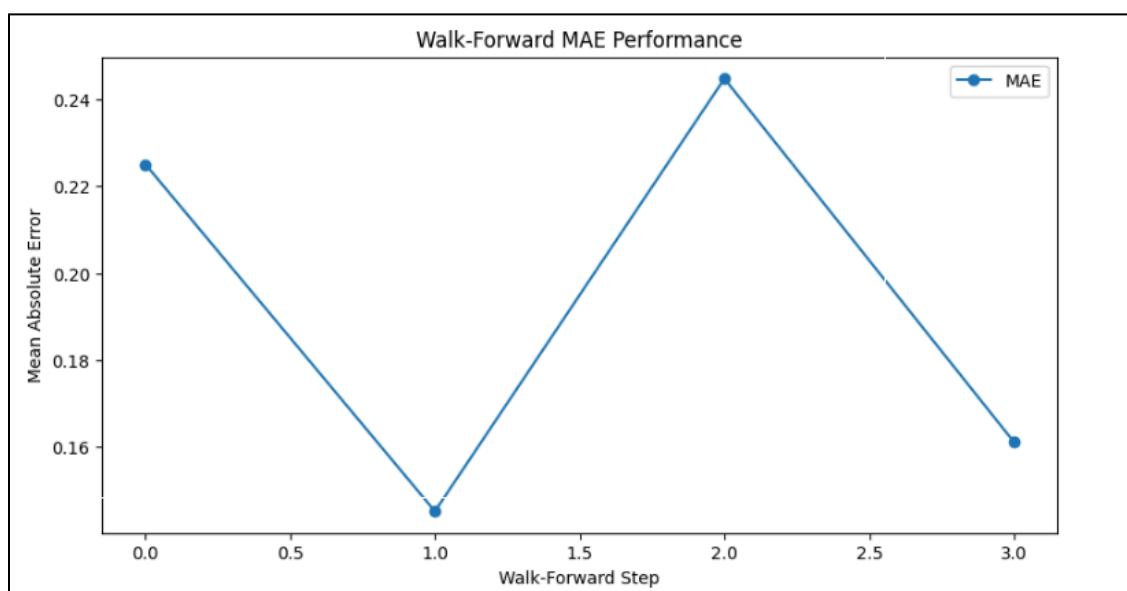
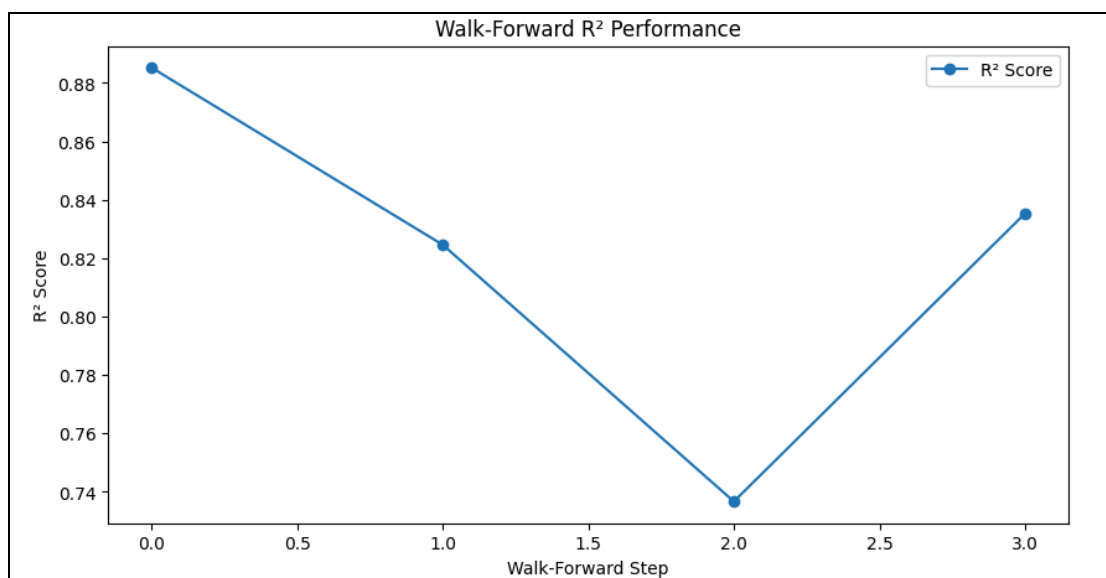


2.1.2 Train Test split: 500:100 for each set

We first scaled the data and then split into train1 and test datasets.

Then, We selected the first 500 records from train1 data for train data and next 100 records for test or validation data as each set for input to the model and after each iteration we move forward by 500 records as step_size.

We selected the model predictions with lowest R-squared on validation data and compared the predictions on test data with actual returns.



Validation Data

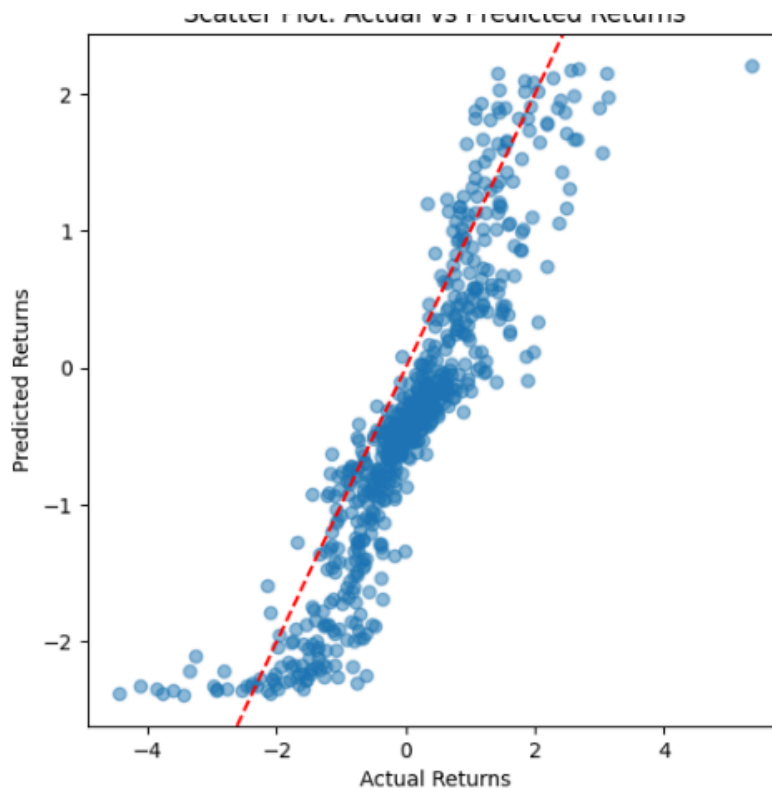
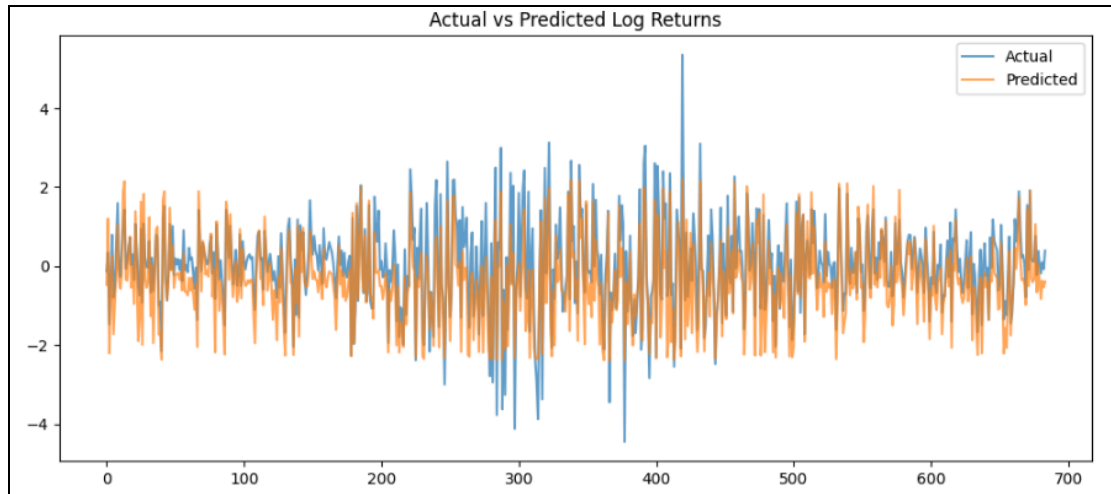
Best R-squared: 0.90

Corresponding MAE: 0.11

Test Data:

Mean Absolute Error (MAE): 0.341

R-squared (R^2): 0.775



2.3 CNN-GAF

2.3.1 Train Test split: 500:500

We converted the features into images and then created subsequences with a window size of 15. We split into train1 and test datasets.

Then, We selected the first 500 records from train1 data for train data and next 500 records for test or validation data as each set for input to the model and after each iteration we moved forward by 500 records as step_size.

We selected the model predictions with highest R-squared on validation data and compared the predictions on test data with actual movement (up or down).

Validation Dataset:

Best R-squared: 0.09750

Corresponding MAE: 0.565

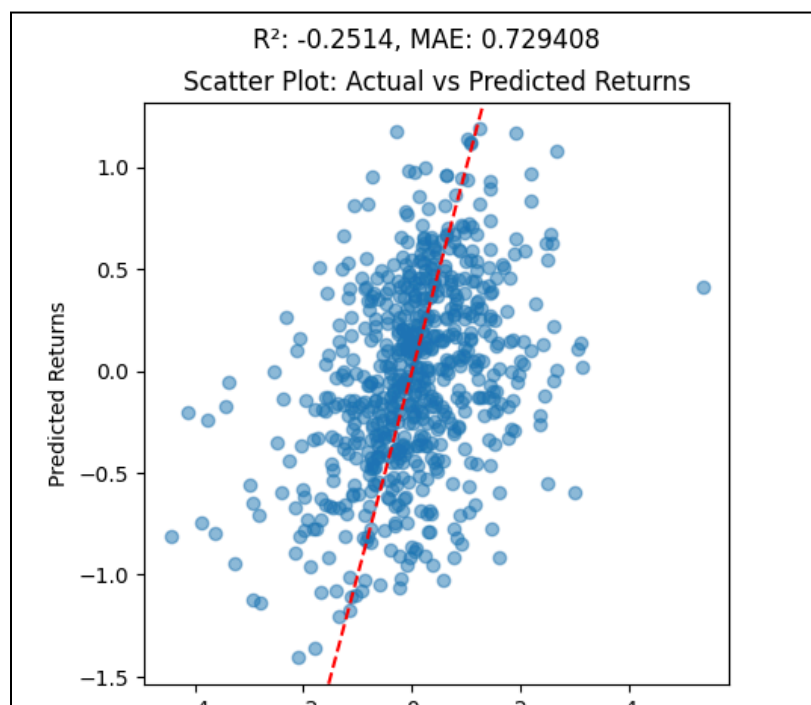
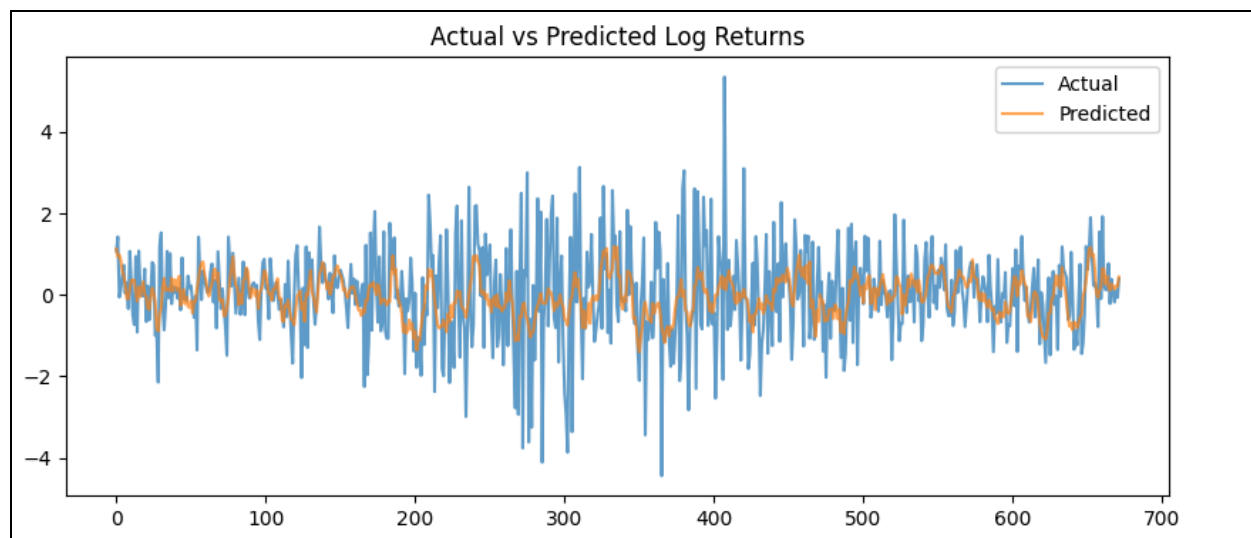
Training Loss: 1.283

Validation Loss: 0.5462

Test Dataset:

Mean Absolute Error (MAE): 0.768

R-squared (R^2): 0.149



2.3.2 Train Test split: 500:100

We converted the features into images and then created subsequences with a window size of 15. We split into train1 and test datasets.

Then, We selected the first 500 records from train1 data for train data and next 100 records for test or validation data as each set for input to the model and after each iteration we move forward by 500 records as step_size.

We selected the model predictions with highest R-squared on validation data and compared the predictions on test data with actual movement (up or down).

Validation Dataset:

Best R-squared: -0.08650

Corresponding MAE: 0.9262

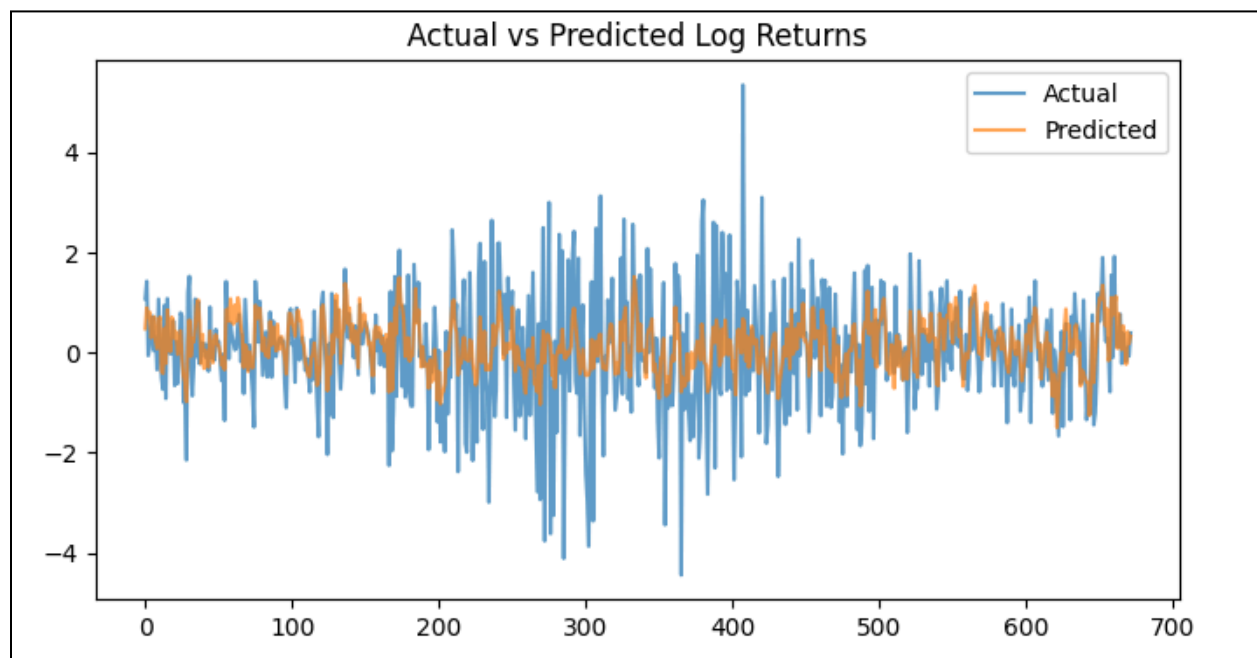
Training Loss: 0.14157

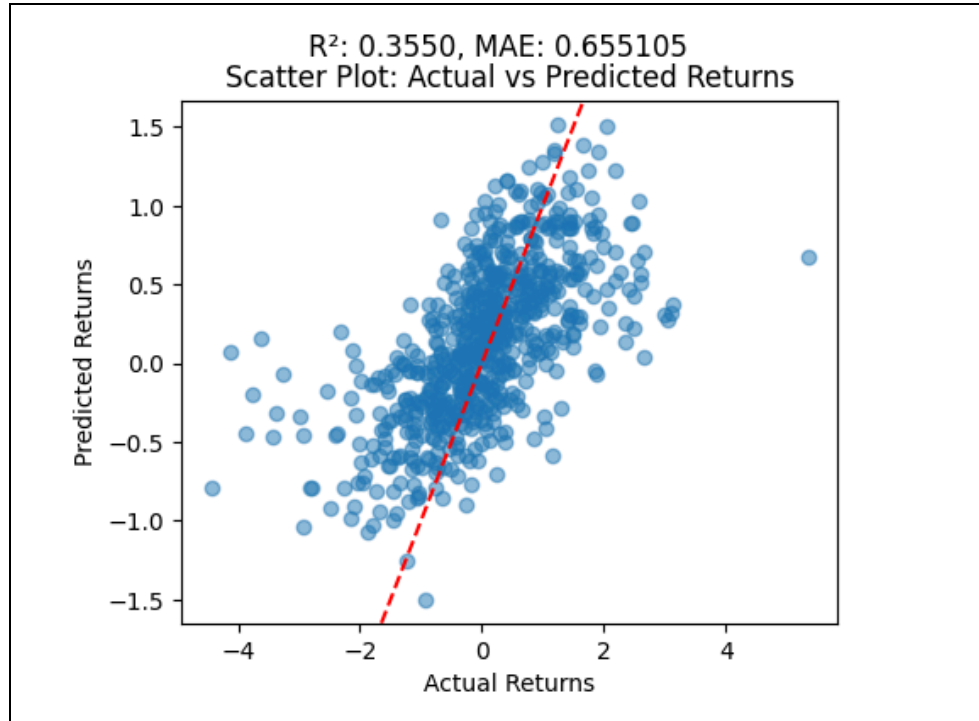
Validation Loss: 0.83588

Test Dataset:

Mean Absolute Error (MAE): 0.65

R-squared (R^2): 0.355





Walk Forward CNN-GAF method has not performed well maybe because no of records are low as large no of iterations 50-60 is need to get to lowloss value

With 3000 records and 500-100 in train-testset, we have stricken a limitation with respect to iterations.

Model Comparison: Regression task

Model:Walk Forward	R-Squared	MAE
MLP Train-Test:500:500	77.5%	0.395
MLP Train-Test:500:100	77.5%	0.34
CNN-GAF Train-Test:500:500	14.9%	0.768
CNN-GAF Train-Test:500:100	35.5.%	0.65

Walk forward CNN-GAF has not performed well, other than that the other model has performed well for the regression tasks with league of information i.e using info from future.

Step 3

3.1 Reducing leakage

We must carefully arrange our train/test splits to prevent future information from impacting past predictions in order to address and limit information leaking. Here are some tactics:

1. Make use of more stringent non-overlapping windows. Make sure the training set doesn't contain any data points from the test set. By carefully choosing the split points so that there is no overlap, this can be accomplished.
2. Establish a Buffer Period To make sure that no future data—which could be interpreted as information from the future—is incorporated into the training process, a buffer time is added between the end of the training data and the beginning of the test data. The buffer interval lowers the possibility of leaking by creating a void during which the model does not see any data.

3.2 Walk Forward LSTM

We downloaded hourly data of SPY from yahoo finance.

Then we created features on hourly lags returns for 10 days i.e. 24*10 features.

Then we try to predict the 3 hour ahead cumulative return that whether is it up (positive) or not (negative) i.e. 0 or 1.

To prevent leakage of information, we have used only past lags and no information from future is used.

There is no overlapping of training and validation data or test data.

Note: <in jupyter the coding for this is under Step2: LSTM>

3.2.1 Train Test split: 500:500 for each set

We split into train1 and test datasets.

Then, We selected the first 500 records from train1 data for train data and next 500 records for test or validation data as each set for input to the model and after each iteration we move forward by 500 records as step_size.

We selected the model predictions with highest accuracy on validation data and compared the predictions on test data with actual movement (up or down).

This is a classification task for this walk forward LSTM case.

Validation DataSet:

Best Accuracy: 1.0

Training Loss: 0.0159

Validation Loss: 0.133

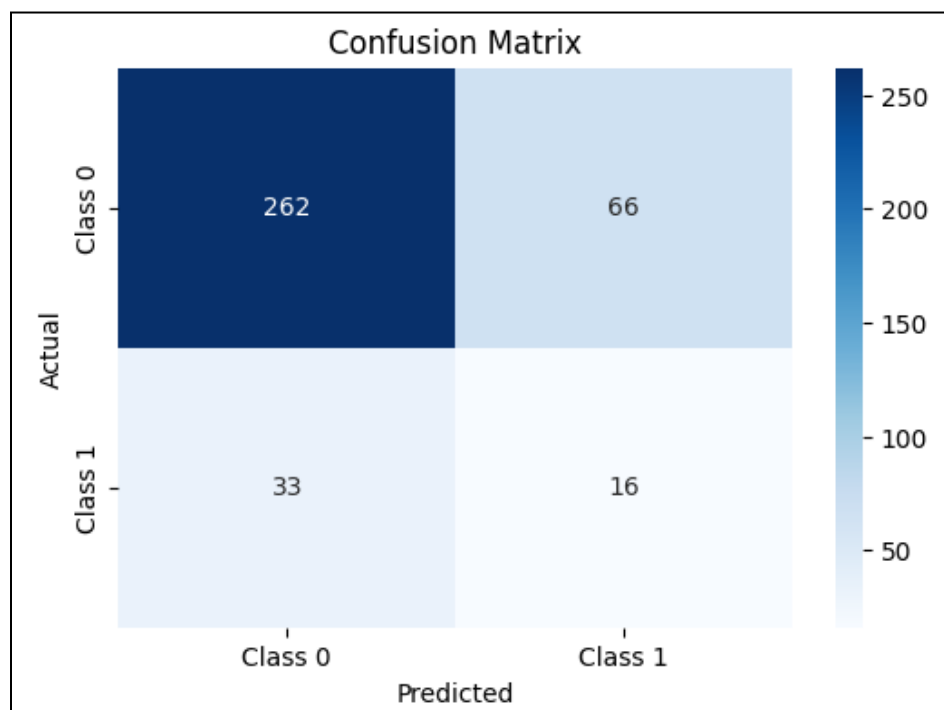
Test Dataset:

accuracy: 0.7374

f1: 0.24

precision: 0.195

recall: 0.32



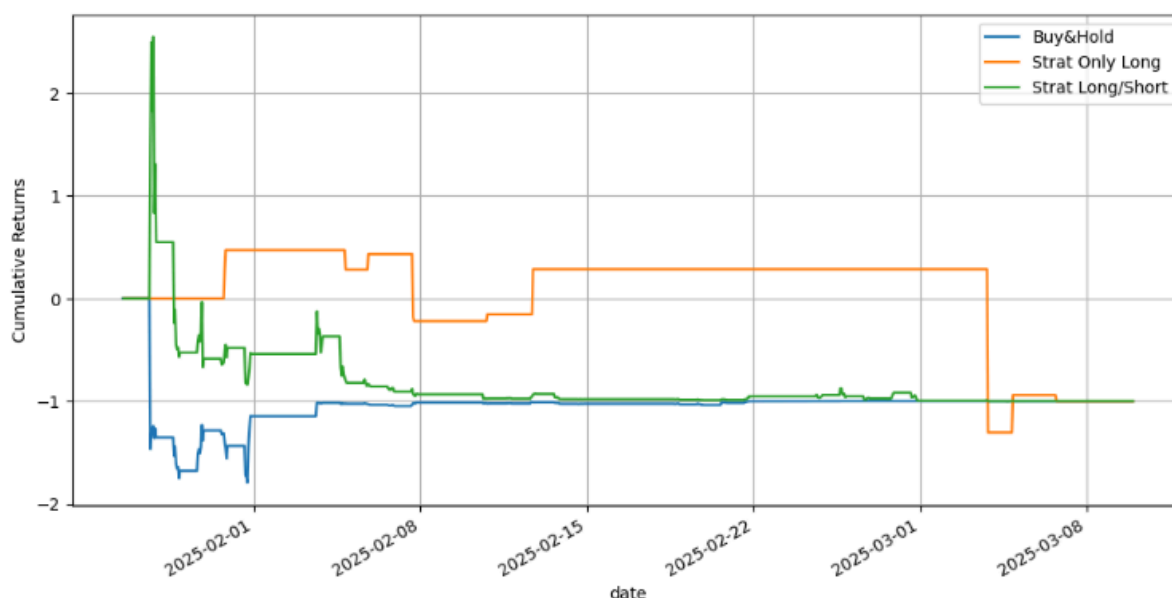
	Class 0	Class 1			
		Predicted			
Classification Report:					
	precision	recall	f1-score	support	
0	0.89	0.80	0.84	328	
1	0.20	0.33	0.24	49	
accuracy			0.74	377	
macro avg	0.54	0.56	0.54	377	
weighted avg	0.80	0.74	0.76	377	

Backtesting:

Long-short strategy: We defined positions as -1 for negative returns and +1 for positive returns

Only Long strategy: We defined positions as 0 for negative returns and +1 for positive returns.

Buy and hold strategy: cumulative returns,



3.2.2 Train Test split: 500:100 for each set

We split into train1 and test datasets.

Then, We selected the first 500 records from train1 data for train data and next 100 records for test or validation data as each set for input to the model and after each iteration we move forward by 500 records as step_size.

We selected the model predictions with highest accuracy on validation data and compared the predictions on test data with actual movement (up or down).

This is a classification task for this walk forward LSTM case.

GROUP WORK PROJECT # 3
Group Number: 8429

MScFE 642: Deep Learning for Finance

Validation Dataset:

Best Accuracy: 1.0

Training Loss: 0.0533

Validation Loss: 0.146

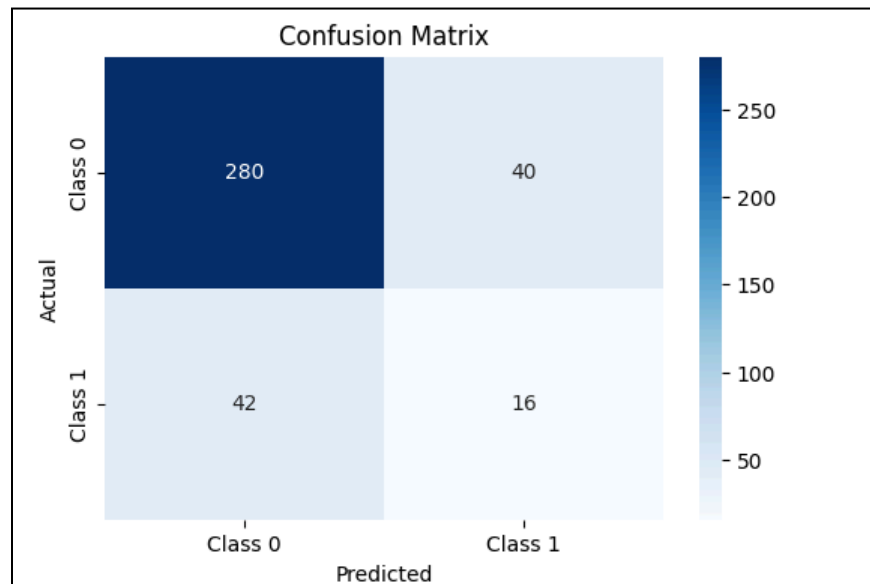
Test Dataset:

accuracy: 0.783

f1: 0.28

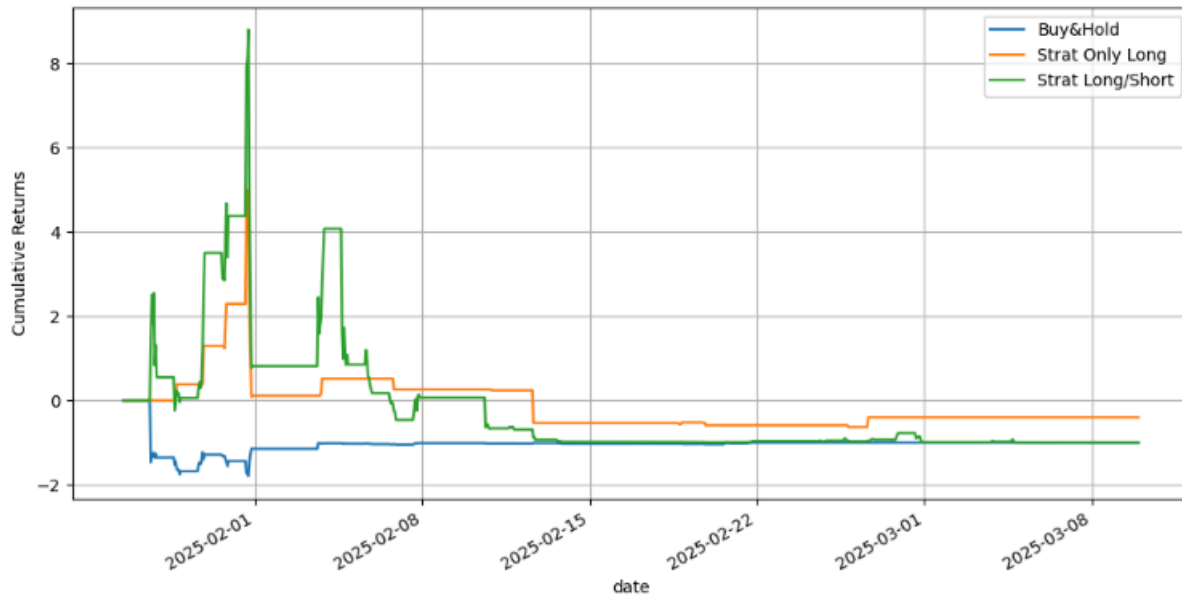
precision: 0.2857

recall: 0.2758



Both the model suffers to pick up class 1's which are low in numbers than class 0s

BackTesting:



Recommend Early exit strategy

Because most of the time the SPY does not have high fluctuations, as it moves within $\pm 2\%$, and this happens due to reversal of short term trend we see reversal of trend, because of which it is difficult to decide which strategy will hold good, long or short.

But it is comparable to a safer asset with low risks and low returns. Those who do not have a risk appetite may invest in this asset.

Model Comparison:

Model: Walk Forward	R-Squared	MAE
MLP Train-Test:500:500	77.5%	0.395
MLP Train-Test:500:100	77.5%	0.34
LSTM Train-Test:500:500	Classification: Accuracy: 74%	
LSTM Train-Test:500:100	Classification Accuracy: 78%	

GROUP WORK PROJECT # 3
Group Number: 8429

MScFE 642: Deep Learning for Finance

CNN-GAF Train-Test:500:500	14.9%	0.768
CNN-GAF Train-Test:500:100	35.5.%	0.65